

التعليقات واستخداماتها

4

التعليقات هي عبارات دخيلة على الشيفرات البرمجية وليست جزءًا منها، إذ تتجاهلها المُصَرِّفات (compilers) والمترجمات (interpreters). يُسهِّل تضمين التعليقات في الشيفرات من قراءتها وفهمها ومعرفة وظيفة كل جزء من أجزائها، لأنها توفر معلومات وشروحات حول ما يفعله كل جزء من البرنامج.

بناءً على الغرض من البرنامج، يمكن أن تكون التعليقات بمثابة مُذَكِّرات لك، أو يمكنك كتابتها لمساعدة المبرمجين الآخرين على فهم الشيفرة. يُستحسن كتابة التعليقات أثناء كتابة البرامج أو تحديثها، لأنك قد تنسى السياق وتسلسل الأفكار لاحقًا، والتعليقات القديمة قد تصبح عديمة الفائدة ومضلة إن لم تُحدَّث.

1. صياغة التعليقات

تبدأ التعليقات السطرية في **بايثون** بالعلامة # ومسافة بيضاء، وتستمر حتى نهاية السطر. بشكل عام، ستبدو التعليقات على النحو التالي:

```
# هذا تعليق
```

نظرًا لأنَّ التعليقات لا تُنفَّذ، فعند تشغيل البرنامج، لن ترى أي إشارة للتعليقات، فالتعليقات توضع في الشيفرة المصدرية ليقرأها الناس، وليس للتنفيذ. في برنامج "Hello, World!" قد يبدو التعليق كما يلي:

```
# طبع "Hello, World!" في سطر الأوامر
print("Hello, World!")
```

في الحلقة `for`، قد تبدو التعليقات كما يلي:

```
# تعريف المتغير sharks كمصفوفة من السلاسل النصية
sharks = ['hammerhead', 'great white', 'dogfish', 'frilled',
          'bullhead', 'requiem']

# sharks حلقه For تمر على المصفوفة
for shark in sharks:
    print(shark)
```

يجب أن تُحاذى التعليقات على نفس المسافة البادئة (indent) للشيفرة التي تعلّق عليها. بمعنى أنّ التعليقات على دالة ليس لها مسافة بادئة ستكون هي أيضًا بدون مسافة بادئة، وسيكون لكل مستوى من المسافات البادئة التالية تعليقات تتوافق مع الشيفرات البرمجية التي تعلّق عليها.

على سبيل المثال، الشيفرة التالية تعرّف الدالة `again()` التي تسأل المستخدم إن كان يريد استخدام الحاسبة مجددًا، مع بعض التعليقات:

```
...
# تعريف الدالة again() لسؤال المستخدم
# إن كان يريد استخدام الحاسبة مجددًا

def again():
    # أخذ المدخلات من المستخدم
    calc_again = input('Do you want to calculate again?
    Please type Y for YES or N for NO.
    ')

    # calculate() الدالة فستُنَفَّذ إذا أدخل المستخدم Y
```

```

if calc_again == 'Y':
    calculate()

# إن كتب المستخدم N فقل وداعا للمستخدم وأنه البرنامج

elif calc_again == 'N':
    print('See you later.')

# إن أدخل المستخدم حرفًا آخر، فأعد تنفيذ الدالة
else:
    again()

```

الهدف من التعليقات هو مساعدة المبرمج الأصلي، أو أي شخص آخر يستخدم مشروعه أو يتعاون معه، على فهمه. وإذا تعدد صيانة التعليقات وتحديثها تحديدًا صحيحًا، وبالتوازي مع الشيفرة، فإنَّ عدم تضمين التعليقات قد يكون أفضل من كتابة تعليق يتناقض مع الشيفرة. يجب أن تجيب التعليقات عن سؤال "لماذا" بدلًا من "ماذا" أو "كيف". لأنَّ ما لم تكن الشيفرة صعبة ومعقَّدة، فإنَّ النظر إلى الشيفرة عمومًا كافٍ لفهم ما الذي تفعله الشيفرة، أو كيف تفعله.

2. التعليقات الكتلية

يمكن استخدام التعليقات الكتلية (Block Comments) لتوضيح الشيفرات البرمجية المعقَّدة التي لا تتوقع أن يكون القارئ على دراية بها. تنطبق هذه التعليقات الطويلة على جزء من الشيفرة أو جميعها، كما توضع في نفس مستوى المسافة البادئة للشيفرة. في التعليقات الكتلية، يبدأ كل سطر بالعلامة # تليها بمسافة بيضاء واحدة. إذا كنت بحاجة إلى استخدام أكثر من فقرة واحدة، فيجب فصلها بسطر يحتوي على علامة # واحدة.

فيما يلي مثال على كتلة تعليقات تشرح ما يحدث في الدالة `main()` المعرفة أدناه:

```
# سوف تحلل الدالة main الوسائط عبر المتغير parser
# الوسائط ستُحدّد بواسطة المستخدم على سطر الأوامر. هذا سيمرر
# الوسيط word الذي يريد المستخدم تحليله مع اسم الملف
# المراد استخدامه، وكذلك تقديم نص مساعد إذا لم يُدخل
# المستخدم الوسيط بشكل صحيح
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "word",
        help="the word to be searched for in the text file."
    )
    parser.add_argument(
        "filename",
        help="the path to the text file to be searched through"
    )
    ...
```

تُستخدَم التعليقات الكتلية عادةً عندما تكون الشيفرة غير واضحة، وتتطلب شرحًا شاملاً. يجب أن تتجنب الإفراط في التعليق على الشيفرة، ويجب أن تثق في قدرة المبرمجين الآخرين على فهم الشيفرة، إلا إذا كنت تكتب لجمهور معين.

3. التعليقات السطرية

توضع التعليقات السطرية (Inline comments) على نفس السطر الذي توجد فيه التعليمة البرمجية. ومثل التعليقات الأخرى، فإنّها تبدأ بالعلامة `#` ومسافة بيضاء واحدة. بشكل عام، تبدو التعليقات السطرية كما يلي:

```
[code] # تعليق مضمن حول الشيفرة
```

لا ينبغي الإكثار من استخدام التعليقات السطرية، ولكن عند استخدامها في محلها يمكن أن تكون فعالة لشرح الأجزاء الصعبة من الشيفرة. وقد تكون مفيدة أيضًا إن ظننت أنك قد لا تتذكر سطرًا من الشيفرة في المستقبل، أو إذا كنت تتعاون مع شخص قد لا يكون على دراية بجميع جوانب الشيفرة.

على سبيل المثال، إذا لم يكن هناك توضيح مسبق، فقد لا تعلم أنت أو المتعاونون معك أن الشيفرة التالية تنشئ عددًا عقديًا، لذلك قد ترغب في إضافة تعليق مضمّن:

```
z = 2.5 + 3j # إنشاء عدد عقدي
```

يمكن أيضًا استخدام التعليقات السطرية لشرح السبب وراء فعل شيء ما، أو بعض المعلومات الإضافية، كما في المثال التالي:

```
x = 8 # ابتداء x بقيمة عشوائية
```

يجب استخدام التعليقات السطرية عند الضرورة وحسب، كما ينبغي أن توفر إرشادات مفيدة للشخص الذي يقرأ البرنامج.

4. تعليق جزء من الشيفرة بدواعي الاختبار والتنقيح

بالإضافة إلى استخدام التعليقات وسيلةً لتوثيق الشيفرة، يمكن أيضًا استخدام العلامة # لتعليق جزء من الشيفرة وتعطيله أثناء اختبار أو تنقيح البرنامج الذي تعمل عليه. أي عندما تواجه أخطاء بعد إضافة أسطر جديدة إلى الشيفرة، فقد ترغب في تعليق بعضها لمعرفة موضع الخلل. يمكن أن يتيح لك استخدام العلامة # تجربة بدائل أخرى أثناء إعداد الشيفرة. على سبيل المثال، قد تفضل بين استخدام الحلقة `while` أو حلقة `for` أثناء برمجة لعبة، ويمكنك تعليق إحداها بينما تختبر أيهما أفضل:

```
import random

number = random.randint(1, 25)

# number_of_guesses = 0

for i in range(5):
    # while number_of_guesses < 5:
        print('Guess a number between 1 and 25:')
        guess = input()
        guess = int(guess)

        # number_of_guesses = number_of_guesses + 1

        if guess < number:
            print('Your guess is too low')

        if guess > number:
            print('Your guess is too high')

        if guess == number:
            break

    if guess == number:
        print('You guessed the number!')

else:
    print('You did not guess the number. The number was ' +
          str(number))
```

يتيح لك تعليق الشيفرة البرمجية تجربة عدّة طرائق ومقاربات برمجية، بالإضافة إلى مساعدتك على العثور على مكن الخطأ من خلال التعليق المنهجي لبعض أجزاء البرنامج.

5. خلاصة الفصل

سيساعدك استخدام التعليقات في برامج **بايثون** على جعل برامجك أكثر مقروئية، سواءً لك أو لغيرك. التعليقات المناسبة وذات الصلة والمفيدة ستسهّل تعاون الآخرين معك في مشاريع البرمجة وتجعل شيفرتك أكثر قيمة.